Développement d'Applications Réparties

Licence en 3ème année Computer Science – GLSI

Dr. Ing. Leïla BAYOUDHI

Maître-Assistante à l'ISIMA

bayoudhi.leila@gmail.com

Développement d'Applications Réparties

Introduction

Séance 1 : 24/09/2026

Système réparti

- Un ensemble d'ordinateurs indépendants qui apparaît à un utilisateur comme un système unique et cohérent :
  - ✓ Les machines sont autonomes.
  - ✓ Les utilisateurs ont l'impression d'utiliser un seul système (**notion de transparence**).
- Un **système réparti** est un **ensemble d'entités autonomes de calcul** (ordinateurs, processeurs, processus, etc.) interconnectées et qui peuvent **communiquer par un réseau de communication**, par moyen d'envoi de messages.

Pourquoi les systèmes répartis?

- ✓ Aspects économiques (rapport prix/performance)
- ✓ Adaptation de la structure d'un système à celle des applications (géographique ou fonctionnelle)
- ✓ Besoin d'intégration (applications existantes initialement séparées)
- ✓ Besoin de communication et de partage d'information
- ✓ Partage de ressources (programmes, données, services)
- ✓ Réalisation de systèmes à haute disponibilité
- ✓ Réalisation de systèmes à grande capacité d'évolution

Application répartie

- Il s'agit d'une application découpée en plusieurs unités :
  - ✓ Chaque unité peut être placée sur une machine différente
  - ✓ Chaque unité peut s'exécuter sur un système différent
  - ✓ Chaque unité peut être programmée dans un langage différent

- Un grand nombre d'applications ne s'exécutent pas intégralement sur un seul nœud de calcul. Il s'agit **d'applications réparties.**

  ➢ **Application répartie = traitements coopérants sur des données réparties.**

© L. Bayoudhi DAR

Distinction entre “système” et “application”

- **Système :** gestion des ressources communes et de l’infrastructure, lié de manière étroite au matériel sous-jacent.
  - Système d’exploitation : gestion de chaque élément.
  - Système de communication : échange d’information entre les éléments.
- **Application :** réponse à un problème spécifique, fourniture des **services** à ses utilisateurs (qui peuvent être d’autres applications).
  - Utilise les **services** généraux fournis par le **système**

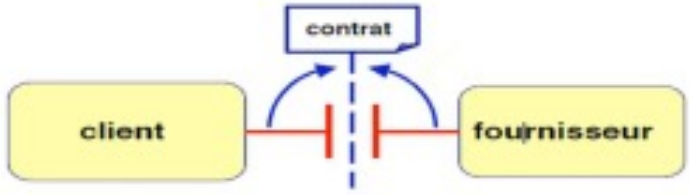
© L. Bayoudhi DAR

Services et Interfaces

Définition

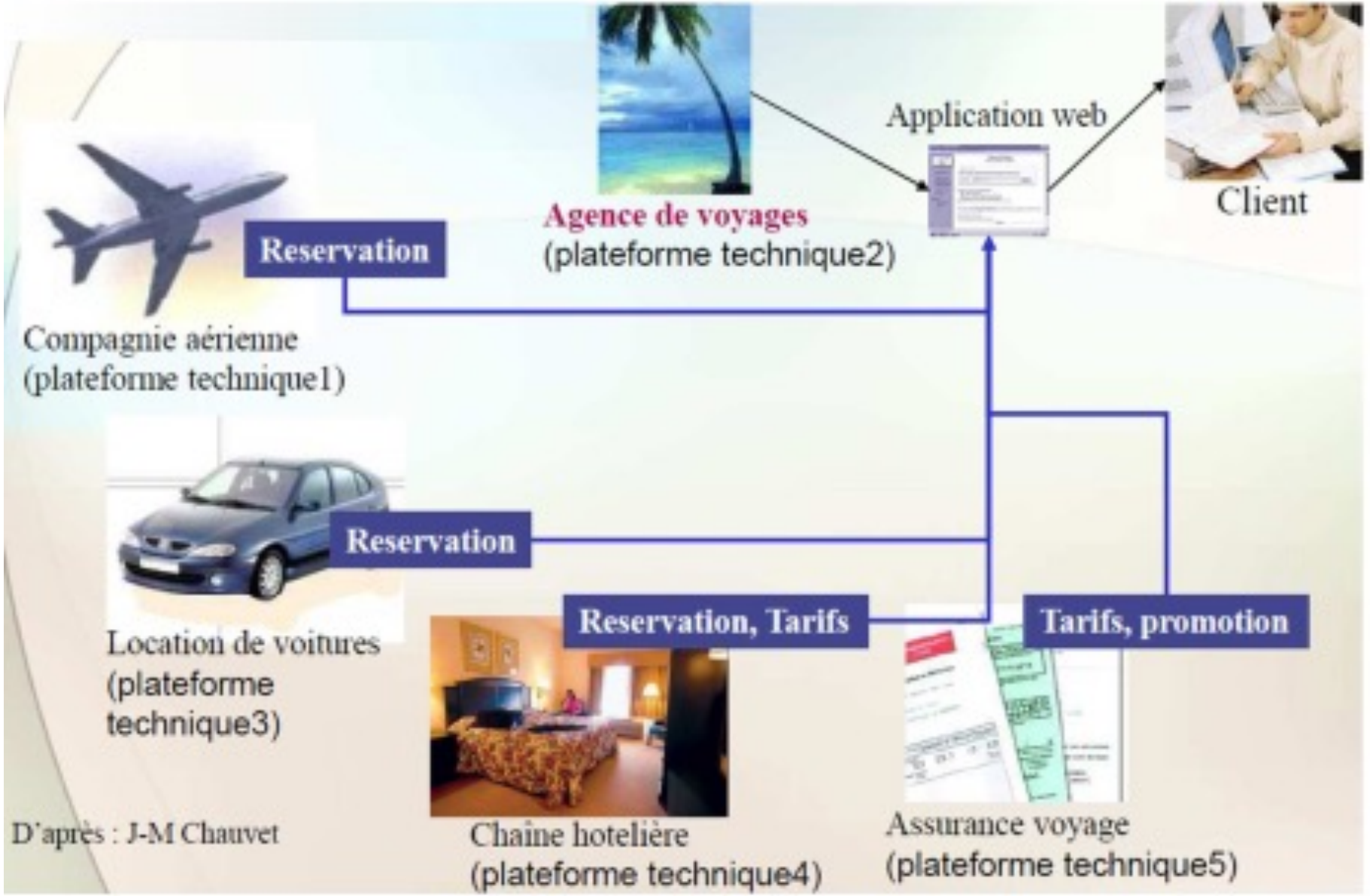
- Un service est un comportement **défini** par **contrat**, qui peut être implémenté et fourni par un composant pour être utilisé par un autre composant, sur la base exclusive du contrat.

Mise en oeuvre



- Un service est **accessible** via une ou plusieurs **interfaces**
- **Une interface** décrit l'interaction entre le demandeur et le fournisseur du service
  - Point de vue opérationnel : définition des opérations et structures de données qui concourent à la réalisation du service.
  - Point de vue contractuel : définition du contrat entre client et fournisseur.

Exemple



© L. Bayoudhi DAR

Exemple

- Un produit « voyage » est la combinaison de plusieurs produits
  - ✓ Gestion de réservation de billets de transport
  - ✓ Gestion de réservation des hôtels
  - ✓ Gestion de réservation de voitures de location
  - ✓ …
- Le résultat des informations récupérées auprès de différents fournisseurs
  - ✓ Compagnies aériennes
  - ✓ Chaînes hôtelières
  - ✓ Agences de location de voitures …

© L. Bayoudhi DAR

Domaines d’applications

- Coordination d’activités et Collaboration
  - ✓ Google Docs, Canva, Jira, Trello
  - ✓ Jeux vidéo en ligne, échecs en ligne
- Communication et partage d’information
  - ✓ le commerce électronique.
  - ✓ Réseaux sociaux (Tiktok, Facebook, Instagram)
  - ✓ WhatsApp, Messenger,
  - ✓ Netflix, YouTube,
- Applications temps réel
  - ✓ Contrôle de procédés industriels, télémédecine
- **Toutes les applications qui nécessitent des utilisateurs ou des données réparties**

Programmation classique versus programmation répartie

- La plupart des applications réparties sont de type **client/serveur** : le client demande des services à un serveur.
  - **En programmation classique**, lorsqu'un programme a besoin d'un service, il appelle localement une fonction/procédure/méthode d'une librairie, d'un objet, etc.
  - **En programmation répartie**, l'appel d'une fonction/procédure/méthode peut se faire à distance.

    ⇨ Proposer des méthodes/concepts/outils permettant de simplifier le développement d'applications réseau client/serveur, en essayant de s'abstraire de l'aspect distant.

Programmation classique versus programmation répartie

Programmation classique	Programmation répartie
● L'utilisateur du service et le fournisseur de service se trouvent sur la même machine : ✓ Même OS ✓ Même espace mémoire ✓ Pas de problème de transport ✓ Disponibilité du service localement (tant que l'on a accès à la librairie) ✓ Un même langage de programmation	● L'utilisateur et le fournisseur de service ne se trouvent pas sur la même machine : deux machines différentes (sans compter celles traversées) : Les utilisateurs ont l'impression sur une seule machine : la transparence ✓ OS différents ✓ Mémoires individuelles ✓ Problème de transport : pare-feu (firewall), réseau, etc. ✓ Retrouver le service ? où se trouve-t-il ? qui le propose ? ✓ Différents langages : paradigmes, passage de paramètres…

Plan

Chapitre I : Sockets

1) Introduction aux sockets
2) Socket en mode connecté (au dessus de TCP)
3) Socket en mode paquet (au dessus de UDP)
4) API Java Java.net

Chapitre II : Architectures client / serveur

1) Problèmes d'intégration d'applications
2) Fondement des architectures client/serveur
3) Système RMI
4) API Java JNDI

Chapitre III : Intergiciels orientés objets (CORBA)

1) introduction aux intergiciels
2) Architecture CORBA
3) Composants et services CORBA
4) Langage IDL et projection en JAVA

Chapitre IV : Intergiciels orientés messages

1) Intergiciels orientés messages
2) API Java JMS
3) Création, manipulation et échange de messages
4) Fiabilité de communication OM